Plan de desarrollo — Simulador orbital (Leyes de Kepler)

Proyecto: Simulador computacional en Python del movimiento orbital bajo gravitación universal, que verifica las tres leyes de Kepler y la conservación de energía y momento angular.

Especificación base: docs/Pre-Proyecto Grupo 6.pdf.

Estado de partida: estructura de carpetas y documentación ya creadas; texturas descargadas; receta de empaquetado lista. No hay código funcional todavía.

Objetivo del plan: llevar el proyecto desde el esqueleto actual hasta un instalador que funcione, con pruebas que demuestren que la física es correcta.

Enfoque: primero el motor físico (cuerpos, gravedad, integradores, conservación), después el análisis (Kepler, gráficas) y al final lo visual (3D, controles) y el empaquetado. Cada fase termina con algo comprobable.

Cómo leer este plan

- Está dividido en **fases numeradas**. Cada fase tiene: **objetivo**, **pasos**, **criterio de cierre** y **riesgos/dependencias**.
- Los pasos están escritos para que se puedan seguir sin ser programador. Cuando aparece un término técnico, se explica en palabras simples.
- **Código vs. documentación:** el código usa nombres en inglés (por ejemplo Body, compute_accelerations) porque así ya están escritos en el esqueleto; todo lo que se lee (comentarios, documentos, mensajes) va en español.
- Los símbolos que se repiten:
- **Prueba**: código que verifica automáticamente que algo funciona.
- **Commit**: guardar un avance en el historial del proyecto (Git).
- **dt**: el "paso de tiempo", cuánto tiempo avanza la simulación en cada ciclo.

Supuestos

Se marcan para poder corregirlos si no aplican:

1. El alcance es el del PDF del pre-proyecto (simulador educativo, dos o más cuerpos, verificación de Kepler).
2. Se usa una sola convención de unidades internas: **AU** (distancia), **masas solares** (masa) y **años** (tiempo). Con eso, la constante gravitacional vale **$G = 4\pi^2$**.
3. El visualizador 3D es **VPython** (ya elegido).
4. La tolerancia de error objetivo es **1 %**.
5. Los tiempos de cada fase son **estimaciones relativas**, no fechas.
6. No se incluye sonido (VPython no reproduce audio).

Resumen

Resultado final (entregables)

1. Programa que simula órbitas y las muestra en 3D.
2. Pruebas automáticas que confirman la física (conservación y leyes de Kepler).
3. Gráficas de análisis (energía, momento angular, área, T^2 vs a^3).
4. Un **instalador/ejecutable** que corre sin tener Python instalado.
5. Informe técnico y material de presentación.

Tecnologías

Uso	Herramienta
Cálculos numéricos	NumPy
Integradores	Código propio (Euler-Cromer y Verlet)
Visualización 3D	VPython
Gráficas 2D	Matplotlib
Pruebas	pytest + pytest-cov
Empaquetado	PyInstaller (+ GitHub Actions para Windows/macOS/Linux)
Control de versiones	Git + GitHub

Reglas fijas del proyecto

- **Separar física de dibujo.** El cálculo nunca se hace dentro de la función que dibuja. Esto permite correr miles de pasos sin depender de los cuadros por segundo.
- **Una sola convención de unidades.** AU, masas solares, años. No mezclar.
- **Validar con números, no con la vista.** Un gráfico bonito no demuestra nada. Se comprueba con energía, momento angular y las leyes de Kepler.
- **Tolerancia 1 %** como meta inicial, confirmable al experimentar con dt.

Estructura de carpetas

Estado actual (ya creada), con lo que cada parte responderá:

```
orbital-simulator/
├── src/
│   ├── physics/          ¿Cómo interactúan los cuerpos?
│   │   ├── bodies.py         Estructura de datos del cuerpo celeste
│   │   ├── gravity.py        Fuerza y aceleración gravitacional
│   │   └── conservation.py   Energía mecánica y momento angular
│   ├── integrators/      ¿Cómo avanza el tiempo?
│   │   ├── euler_cromer.py
│   │   └── verlet.py
│   ├── kepler/           ¿Se cumplen las leyes?
│   │   └── kepler_laws.py
│   ├── render/           ¿Qué ve el usuario?
│   │   ├── scene3d.py        Escena 3D (VPython)
│   │   └── plots.py          Gráficas 2D (Matplotlib)
│   ├── ui/               Entradas del usuario
│   │   └── controls.py
│   └── main.py           Orquestador (une todo)
├── tests/                Pruebas unitarias y de escenarios
├── data/                 presets.json (condiciones iniciales)
├── docs/                 informe, guías y este plan
├── assets/textures/      Texturas de planetas, luna, sol y fondo
├── scripts/              Utilidades (descarga de texturas, PDF)
├── build_hooks/          Hook de PyInstaller para VPython
├── orbital_simulator.spec    Receta del instalador
└── .github/workflows/    Compilación automática (Windows/macOS/Linux)
```

Fase 1 — Preparar el entorno y confirmar la estructura

Objetivo: tener un lugar de trabajo donde el proyecto se pueda ejecutar y probar sin errores de instalación.

Pasos

1. Instalar Python 3.11 o superior (si no está).
2. Crear un "entorno virtual" (una carpeta aislada con las librerías del proyecto, para no mezclarlas con las del sistema).
3. Instalar las dependencias: pip install -r requirements-dev.txt.
4. Verificar que las librerías cargan: python -c "import numpy, matplotlib, vpython, pytest".
5. Confirmar que la estructura de carpetas existe y está vacía de lógica.
6. Primer commit si hay cambios.

Criterio de cierre

- pytest se ejecuta sin errores de instalación (aunque no haya pruebas reales todavía).
- Los cuatro paquetes importan correctamente.

Riesgos / dependencias

- **Dependencia:** ninguna. Es la base de todo.
- **Riesgo:** entornos con Python viejo. Mitigación: verificar la versión antes.

Fase 2 — Cuerpos celestes (bodies.py)

Objetivo: definir qué es un cuerpo (masa, posición, velocidad, nombre) sin hacer cálculos.

Pasos

1. Escribir una prueba que cree un cuerpo y compruebe que guarda sus datos.
2. Correr la prueba y ver que falla (todavía no existe el código).
3. Crear la clase Body con: name, mass, position, velocity, acceleration. Posición y velocidad como vectores de NumPy de 3 números.
4. Correr la prueba y ver que pasa.
5. Commit.

Criterio de cierre

- Se puede crear un cuerpo con masa, posición y velocidad, y leer esos valores.
- La prueba de bodies pasa.

Riesgos / dependencias

- **Dependencia:** Fase 1.
- **Riesgo:** mezclar unidades. Mitigación: documentar AU / masas solares / años.

Fase 3 — Gravedad (gravity.py)

Objetivo: calcular la fuerza y la aceleración entre cuerpos con la Ley de Gravitación Universal.

Pasos

1. Definir la constante $G = 4\pi^2$ en las unidades del proyecto.
2. Escribir una prueba con dos cuerpos fijos y comparar la aceleración contra un valor calculado a mano.
3. Correr la prueba y ver que falla.
4. Implementar compute_accelerations(bodies): para cada cuerpo, sumar la atracción de todos los demás usando la fórmula vectorial $a_i = G \cdot \Sigma [m_j \cdot (r_j - r_i) / |r_j - r_i|^3]$.
5. Correr la prueba y ver que pasa.
6. Commit.

Criterio de cierre

- La aceleración coincide con el cálculo manual en un caso simple.
- La fuerza atrae (no empuja).

Riesgos / dependencias

- **Dependencia:** Fase 2.
- **Riesgo:** división por cero si dos cuerpos coinciden. Mitigación: distancia mínima de seguridad. Este cálculo es el corazón del simulador: si está mal, todo lo demás falla.

Fase 4 — Integrador Euler-Cromer

Objetivo: avanzar el estado en el tiempo con el método más simple.

Pasos

1. Escribir una prueba de una órbita circular de dos cuerpos: tras un periodo, el cuerpo debe volver cerca del punto de partida.
2. Correr la prueba y ver que falla.
3. Implementar step(bodies, dt): primero actualizar la velocidad con la aceleración, después la posición con la velocidad nueva.
4. Correr la prueba y ver que pasa.
5. Imprimir coordenadas de una órbita para confirmar que "cierra".
6. Commit.

Criterio de cierre

- Una órbita circular se mantiene cerrada (dentro de la tolerancia).

Riesgos / dependencias

- **Dependencia:** Fase 3.
- **Riesgo:** dt demasiado grande hace "explotar" la órbita. Mitigación: empezar con dt pequeño y aumentarlo probando.

Fase 5 — Conservación (conservation.py)

Objetivo: medir energía mecánica y momento angular, y usarlos como indicadores de error.

Pasos

1. Escribir pruebas con valores conocidos de energía y momento angular.
2. Correr y ver que fallan.
3. Implementar kinetic_energy, potential_energy, mechanical_energy, angular_momentum y relative_error.
4. Correr y ver que pasan.
5. Agregar una prueba de órbita circular: la variación de energía y de momento angular debe quedar dentro del 1 %.
6. Commit.

Criterio de cierre

- Energía y momento angular se calculan bien y se mantienen casi constantes en una órbita circular.

Riesgos / dependencias

- **Dependencia:** Fase 4.
- **Riesgo:** que se disparen indica error en el integrador o en la gravedad. Esta fase es el detector de errores del proyecto.

Fase 6 — Integrador Verlet y comparación

Objetivo: añadir un segundo método y comparar cuál conserva mejor la energía.

Pasos

1. Escribir una prueba de que Verlet avanza el estado y conserva la energía igual o mejor que Euler-Cromer.
2. Correr y ver que falla.
3. Implementar step(bodies, dt) con la fórmula posicional de Verlet, manejando el primer paso (necesita el estado anterior).
4. Correr y ver que pasa.
5. Ejecutar la misma órbita con ambos métodos y comparar la variación de energía.
6. Commit.

Criterio de cierre

- Los dos integradores funcionan y existe una comparación con datos.

Riesgos / dependencias

- **Dependencia:** Fase 5.
- **Riesgo:** el arranque de Verlet es la parte delicada. Mitigación: probar solo el primer paso.

Fase 7 — Historial y gráficas (plots.py)

Objetivo: guardar el estado de cada paso y producir las gráficas de análisis.

Pasos

1. En main.py, crear el historial: listas de tiempo, posición, energía y momento angular.
2. Implementar las gráficas con Matplotlib: energía vs. tiempo, momento angular vs. tiempo, área barrida, y T^2 vs a^3.
3. Verificar que cada gráfica sale con sus ejes y rótulos.
4. Guardar las figuras en una carpeta de resultados (ignorada por Git).
5. Commit.

Criterio de cierre

- Se obtienen las cuatro gráficas del PDF (sección 6.2) a partir de una corrida.

Riesgos / dependencias

- **Dependencia:** Fase 6.
- **Riesgo:** creer que la gráfica bonita ya valida la física. Mitigación: la gráfica es evidencia, no prueba; la prueba son los números.

Fase 8 — Verificación de las leyes de Kepler (kepler_laws.py)

Objetivo: comprobar las tres leyes con los datos del historial.

Pasos

1. **Primera ley:** calcular periastro y apoastro (punto más cercano y más lejano), el semieje mayor a = (r_peri + r_apo)/2 y la excentricidad e = (r_apo − r_peri)/(r_apo + r_peri). Confirmar que la órbita ligada es cerrada y ajusta una elipse.
2. **Segunda ley:** calcular el área barrida por el radio en intervalos de tiempo iguales; comprobar que es casi constante.
3. **Tercera ley:** obtener el periodo T y el semieje a de varias órbitas alrededor del

mismo cuerpo central; comprobar que T^2/a^3 se mantiene constante.
4. Escribir pruebas para cada ley y hacerlas pasar.
5. Commit.

Criterio de cierre

- Las tres leyes se verifican dentro de la tolerancia (1 %).

Riesgos / dependencias

- **Dependencia:** Fase 7.
- **Riesgo:** pocas órbitas para la tercera ley dan una constante poco fiable. Mitigación: usar al menos tres radios distintos.

Fase 9 — Escena 3D y texturas (scene3d.py)

Objetivo: mostrar el sistema en 3D leyendo el estado, sin calcular física.

Pasos

1. Crear las esferas de los cuerpos (Sol con brillo propio; planetas con color y textura desde assets/textures/).
2. Dibujar las trayectorias con curvas y los ejes de referencia.
3. Poner el fondo estelar (puntos o textura de la Vía Láctea).
4. Aplicar un factor de escala visual y **dejar claro que el tamaño en pantalla no es el físico**.
5. Actualizar la escena cada N pasos, no en cada paso.
6. Esperar a que carguen las texturas (scene.waitfor("textures")).
7. Commit.

Criterio de cierre

- La escena muestra los cuerpos con textura y la trayectoria, sin afectar la velocidad del cálculo.

Riesgos / dependencias

- **Dependencia:** Fase 8 (y las texturas ya descargadas).
- **Riesgo:** invertir demasiado tiempo en estética antes de tener la física. Mitigación: esta fase va después de validar Kepler.

Fase 10 — Controles de la interfaz (controls.py)

Objetivo: permitir cambiar masa y velocidad inicial desde la pantalla.

Pasos

1. Añadir controles (deslizadores y botones) con las herramientas nativas de VPython: iniciar, pausar, reiniciar.
2. Calcular y **mostrar la velocidad de escape**, para que el usuario sepa cuándo la órbita pasa a ser abierta (hiperbólica).
3. Conectar los controles con el estado inicial que usa main.py.
4. Probar que cambiar un valor cambia la órbita.
5. Commit.

Criterio de cierre

- El usuario cambia los valores y ve el resultado, sin tocar el código.

Riesgos / dependencias

- **Dependencia:** Fase 9.
- **Riesgo:** confundir la interfaz con la física. Mitigación: los controles solo cambian

valores; no calculan nada.

Fase 11 — Escenarios de prueba

Objetivo: demostrar con datos que cada tipo de órbita aparece cuando corresponde.

Pasos

1. **Circular:** velocidad igual a la circular; radio casi constante.
2. **Elíptica:** velocidad entre 0 y la circular; la distancia al foco varía.
3. **Hiperbólica:** velocidad mayor que la de escape; la distancia crece sin volver.
4. Escribir una prueba por escenario y hacerlas pasar.
5. Commit.

Criterio de cierre

- Los tres escenarios pasan sus pruebas con la tolerancia definida.

Riesgos / dependencias

- **Dependencia:** Fase 10.
- **Riesgo:** el escenario hiperbólico es el que más falla si la velocidad inicial no supera la de escape. Mitigación: usar la fórmula de escape.

Fase 12 — Empaquetado e instalación

Objetivo: generar un instalador que funcione sin tener Python instalado.

Pasos

1. Añadir las texturas a la receta orbital_simulator.spec (para que se incluyan) y resolver la ruta correctamente cuando el programa está empaquetado.
2. Compilar en local: pyinstaller orbital_simulator.spec.
3. Probar el ejecutable en una carpeta distinta (o en otro computador).
4. Publicar una etiqueta (git tag v0.1.0) para que GitHub Actions genere las versiones de Windows, macOS y Linux.
5. Descargar y probar cada paquete.
6. Commit.

Criterio de cierre

- El ejecutable abre la simulación en un computador sin Python.

Riesgos / dependencias

- **Dependencia:** Fase 11.
- **Riesgo:** PyInstaller no compila entre sistemas operativos. Mitigación: usar el workflow de GitHub Actions.
- **Riesgo:** VPython necesita archivos internos (JavaScript/WebGL). Mitigación: el hook build_hooks/hook-vpython.py.

Fase 13 — Informe, validación final y presentación

Objetivo: cerrar el proyecto con evidencia y material de exposición.

Pasos

1. Completar docs/informe_tecnico.md con resultados, gráficas y tablas.
2. Reportar el error porcentual observado en conservación y en Kepler.
3. Registrar el crédito de las texturas (CC BY 4.0).
4. Ensayar la demostración: abrir el instalador, cambiar velocidades y mostrar los tres

tipos de órbita.
5. Guardar copia de respaldo y revisar que todo quede subido a GitHub.
6. Commit final.

Criterio de cierre

- Informe completo, simulador instalable y demostración ensayada.

Riesgos / dependencias

- **Dependencia:** Fase 12.
- **Riesgo:** presentar sin datos de validación. Mitigación: el informe se llena con las pruebas y gráficas ya obtenidas.

Cronograma sugerido (estimaciones relativas)

Fase	Tema	Esfuerzo aproximado
1	Entorno y estructura	Muy bajo
2	Cuerpos	Bajo
3	Gravedad	Bajo
4	Euler-Cromer	Bajo
5	Conservación	Medio
6	Verlet y comparación	Medio
7	Historial y gráficas	Medio
8	Leyes de Kepler	Alto
9	Escena 3D y texturas	Medio
10	Controles	Medio
11	Escenarios	Medio
12	Empaquetado	Medio
13	Informe y presentación	Medio

Las fases 1 a 8 son la parte crítica: si el motor físico no está validado, lo visual no sirve. Las fases 9 a 13 se pueden solapar una vez la física es estable.

Definición de "terminado"

Un paso está terminado cuando:

1. Existe su prueba automática y pasa.
2. Produce el resultado esperado dentro de la tolerancia.
3. Está guardado con un commit.

Riesgos generales del proyecto

Riesgo	Probabilidad	Impacto	Mitigación
dt mal elegido (órbita inestable)	Alta	Alto	Probar varios dt; priorizar precisión sobre velocidad
Confundir unidades	Media	Alto	Una sola convención documentada
Acoplar física y dibujo	Media	Medio	El cálculo va aparte del dibujo
Resultado incorrecto que "se ve bien"	Media	Alto	Validar energía, momento angular y Kepler

Riesgo	Probabilidad	Impacto	Mitigación
Pérdida de datos	Media	Alto	Git, GitHub y commits frecuentes

Anexo A — Interfaces de código (nombres y firmas)

Decisiones ya fijadas para que el código de cada fase encaje:

Archivo	Elemento	Entrada → Salida
physics/bodies.py	Body	name, mass, position, velocity, acceleration
physics/gravity.py	compute_accelerations(bodies)	cuerpos → lista de aceleraciones
physics/conservation.py	mechanical_energy(bodies)	cuerpos → energía total
physics/conservation.py	angular_momentum(bodies)	cuerpos → vector momento angular
physics/conservation.py	relative_error(valor, inicial)	dos números → porcentaje
integrators/euler_cromer.py	step(bodies, dt)	estado + paso → nuevo estado
integrators/verlet.py	step(bodies, dt)	estado + paso → nuevo estado
kepler/kepler_laws.py	check_first_law(positions)	trayectoria → semieje y excentricidad
kepler/kepler_laws.py	check_second_law(positions, dt)	trayectoria → áreas por intervalo
kepler/kepler_laws.py	check_third_law(orbits)	varias órbitas → relación T^2/a^3
render/scene3d.py	update(history_slice)	estado → escena dibujada
render/plots.py	plot_energy(history)	historial → figura
ui/controls.py	controles	interacción → estado inicial
main.py	main()	— → simulación completa

Anexo B — Créditos de recursos

Texturas planetarias y fondo: **Solar System Scope** (https://www.solarsystemscope.com/textures/), licencia **CC BY 4.0**. Detalle completo en docs/recursos_visuales.md.